

Database Indexing

Database Indexing is a data structure technique that improves the speed of data retrieval operations on a database table at the cost of additional storage space and slower writes.

Core Concept

An index is like a book's index or a library's card catalog. Instead of scanning every page to find a topic (or every row to find data), you look up the index which tells you exactly where to find what you need.

How Indexes Work

Basic structure: An index creates a separate data structure that stores:

1. The indexed column value(s)
2. A pointer/reference to the actual row location

Example:

Users Table (without index):

id	name	email	age
1	Charlie	charlie@email.com	30
2	Alice	alice@email.com	25
3	Bob	bob@email.com	35
4	Alice	alice2@email.com	28

Index on 'name' (simplified):

Alice → [rows 2, 4]

Bob → [row 3]

Charlie → [row 1]

Index Advantages

Faster queries Dramatic speed improvement for searches, especially on large tables.

Efficient sorting `ORDER BY` on indexed column is fast.

Faster joins Indexes on foreign keys speed up JOIN operations.

Unique constraints Enforce data integrity while providing performance benefits.

Optimized aggregations MIN/MAX queries on indexed columns are instant.

Index Disadvantages

Slower writes Every INSERT, UPDATE, DELETE must also update all relevant indexes.

Storage overhead Indexes consume disk space—sometimes as much as the table itself.

Maintenance cost Indexes need to be maintained, rebuilt, analyzed.

Too many indexes Can confuse the query optimizer and slow down overall performance.

Memory usage Database tries to keep frequently used indexes in memory.

When to Create Indexes

Create indexes on:

Primary keys

- Usually automatic
- Ensures uniqueness and fast lookups

Foreign keys

- Dramatically speeds up JOINS
- Often forgotten but very important

Columns in WHERE clauses

- Frequently filtered columns
- WHERE user_id = 123, WHERE status = 'active'

Columns in ORDER BY

- Frequently sorted columns
- ORDER BY created_at DESC

Columns in JOIN conditions

- Both sides of JOIN should be indexed
- JOIN orders ON orders.user_id = users.id

Columns in GROUP BY

- Aggregation queries
- GROUP BY category

Frequently searched text fields

- Full-text indexes for content search

When NOT to Create Indexes

Don't index:

Small tables

- Tables with <1000 rows—sequential scan is fast enough

Columns with low cardinality

- Boolean columns (true/false only)
- Gender columns (M/F/Other)
- Exception: If you frequently query a specific value (partial index)

Columns that are frequently updated

- Index maintenance overhead exceeds benefits

Wide columns

- Very large text blobs
- Use full-text indexes instead if needed

Every column

- More indexes != better performance
- Slows down writes significantly

TYPES OF INDEXES

1. Single-Column Index

Index on one column.

```
CREATE INDEX idx_email ON users(email);
```

2. Composite (Multi-column) Index

Index on multiple columns.

```
CREATE INDEX idx_name_age ON users(name, age);
```

Best practice: Put the most selective column first, or the column used in WHERE clauses most frequently.

```
sql

-- Uses index fully:
SELECT * FROM users WHERE city = 'New York' AND age = 25;

-- Uses index partially (only city part):
SELECT * FROM users WHERE city = 'New York';

-- CANNOT use index (age is not the first column):
SELECT * FROM users WHERE age = 25;
```

Example with three columns:

```
sql

CREATE INDEX idx_orders ON orders(user_id, status, created_at);

-- Excellent (uses all three):
WHERE user_id = 123 AND status = 'pending' AND created_at > '2024-01-01'

-- Good (uses user_id, status):
WHERE user_id = 123 AND status = 'pending'

-- Good (uses user_id):
WHERE user_id = 123

-- Poor (cannot use index):
WHERE status = 'pending' AND created_at > '2024-01-01'
...
```

3. Unique Index

Ensures no duplicates.

```
CREATE UNIQUE INDEX idx_email_unique ON users(email);
```

Benefits:

- Data integrity (no duplicate emails)
- Index performance (fast lookups)

4. Primary Index

Automatically created for the primary key.

```
PRIMARY KEY (id)
```

5. Full-text Index

Used for text search.

`FULLTEXT(name, description)`

-- PostgreSQL

```
CREATE INDEX idx_articles_content_fts
ON articles USING GIN (to_tsvector('english', content));
```

Best for:

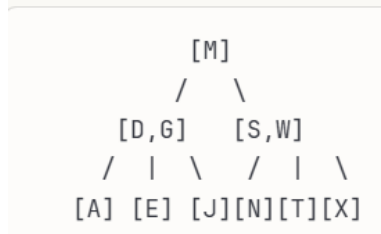
- Blog posts, articles, documents
- Product descriptions
- Any text search functionality

6. B-Tree Index (Most Common)

Balanced tree structure that keeps data sorted and allows searches, insertions, and deletions in logarithmic time. It is the default index type in PostgreSQL, MySQL, Oracle, SQL Server.

```
CREATE INDEX idx_users_age ON users(age);
```

Structure:



Best for:

- Range queries: `WHERE age BETWEEN 25 AND 35`
- Equality searches: `WHERE id = 100`
- Sorting: `ORDER BY created_at`
- Prefix matching: `WHERE name LIKE 'Ali%'`

7. Hash Index

Uses hash function to map keys to bucket locations. Extremely fast for exact matches.

```
CREATE INDEX idx_users_email_hash ON users USING HASH (email);
```

Best for:

- Exact equality searches: `WHERE email = 'alice@email.com'`
- Very fast $O(1)$ lookups

Limitations:

- Cannot handle range queries
- Cannot use for sorting
- Not useful for `LIKE` queries

8. Bitmap Index

Uses bitmaps (arrays of bits) to represent which rows contain each value.

```
**Example:**
...

Gender column (only 3 values: M, F, Other)

Bitmap for 'M':      [1, 0, 1, 1, 0, 0, 1, ...]
Bitmap for 'F':      [0, 1, 0, 0, 1, 1, 0, ...]
Bitmap for 'Other':  [0, 0, 0, 0, 0, 0, 0, ...]

Row 1: M (bit 1 set in M bitmap)
Row 2: F (bit 1 set in F bitmap)
Row 3: M (bit 1 set in M bitmap)
```

Best for:

Columns with low cardinality (few distinct values)

Data warehouse/analytics workloads

Complex boolean logic: WHERE gender = 'F' AND status = 'active'

Limitations:

Poor for high-cardinality columns

Slow for frequent updates (entire bitmap needs updating)

Not available in PostgreSQL/MySQL (mainly Oracle, specialized DBs)

9. Spatial Index (R-Tree, GiST)

Index for geometric or geographic data (points, lines, polygons).

-- PostgreSQL with PostGIS

```
CREATE INDEX idx_locations_point ON locations USING GIST (coordinates);
```

Best for:

- Geographic queries (find nearby locations)
- Mapping applications
- Spatial relationships (overlaps, contains, within)

10. Partial Index

Index on a subset of rows matching a condition.

```
sql

-- Only index active users
CREATE INDEX idx_active_users ON users(email) WHERE status = 'active';

-- This query uses the index:
SELECT * FROM users WHERE email = 'alice@email.com' AND status = 'active';

-- This query does NOT use the index:
SELECT * FROM users WHERE email = 'alice@email.com' AND status = 'inactive';
```

Benefits:

- Smaller index size
- Faster index maintenance
- Better performance for common queries

Best for:

- When you frequently query a specific subset of data
- Columns with skewed distributions

11. Expression/Functional Index

Index on the result of a function or expression rather than raw column value.

```
sql

-- Index lowercase version of email for case-insensitive searches
CREATE INDEX idx_users_email_lower ON users(LOWER(email));

-- This uses the index:
SELECT * FROM users WHERE LOWER(email) = 'alice@email.com';

-- Another example: index on extracted year
CREATE INDEX idx_orders_year ON orders(EXTRACT(YEAR FROM created_at));
SELECT * FROM orders WHERE EXTRACT(YEAR FROM created_at) = 2024;
```

Best for:

- Case-insensitive searches
- Date/time extractions
- Computed values frequently queried

12. Covering Indexes

Index that contains all columns needed for a query, so the database doesn't need to access the table at all.

```
sql

-- Query we want to optimize:
SELECT name, email FROM users WHERE city = 'New York';

-- Create covering index:
CREATE INDEX idx_users_city_covering ON users(city, name, email);
```

The index contains city (for WHERE), name and email (for SELECT), so the database can satisfy the entire query from the index alone—this is called an "index-only scan."

Benefits:

- Extremely fast (no table access needed)
- Reduced I/O

Drawbacks:

- Larger index size
- Slower writes (more data to index)

Clustered & Non-clustered Indexes

Clustered Index: The table data itself is stored in the order of the index. There can only be **one** clustered index per table because data can only be physically sorted one way. Table rows are physically ordered by the clustered index key.

Benefits

- Very fast range queries on the clustered key
- Related data stored together physically
- No separate index structure needed for the key itself

Drawbacks

- Insertions can be slow (may need to reorganize physical storage)
- Only one per table

Non-clustered Index: Separate structure from the table data. Can have many per table.

Benefits:

- Can have multiple non-clustered indexes
- Flexible - index any column(s)

Drawbacks:

- Requires extra lookup (index → pointer → actual row)
- Takes additional storage space

Index Maintenance

Analyze/Update statistics:

```
sql

-- PostgreSQL
ANALYZE users;

-- MySQL
ANALYZE TABLE users;
```

Helps query planner make better decisions.

Rebuild fragmented indexes:

```
sql

-- PostgreSQL
REINDEX INDEX idx_users_email;

-- MySQL
OPTIMIZE TABLE users;
```

Over time, indexes become fragmented (especially with many updates/deletes).

Monitor index usage:

```
sql

-- PostgreSQL: Find unused indexes
SELECT schemaname, tablename, indexname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
AND indexrelname NOT LIKE 'pg_toast%';
```

Remove indexes that aren't being used.

Query Optimization with Indexes

Check if index is used:

```
-- PostgreSQL
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'alice@email.com';

-- Look for:
-- "Index Scan using idx_users_email" (good)
-- "Seq Scan on users" (bad - full table scan)
```

Force index usage (MySQL):

```
SELECT * FROM users USE INDEX (idx_users_email) WHERE email = 'alice@email.com';
```

Avoid index-killing patterns:

Functions on indexed columns:

```
-- Does NOT use index on email:
SELECT * FROM users WHERE UPPER(email) = 'ALICE@EMAIL.COM';

-- Uses index (create functional index):
CREATE INDEX idx_users_email_upper ON users(UPPER(email));
SELECT * FROM users WHERE UPPER(email) = 'ALICE@EMAIL.COM';
```

Leading wildcards:

```
-- Does NOT use index:
SELECT * FROM users WHERE email LIKE '%@gmail.com';

-- Uses index:
SELECT * FROM users WHERE email LIKE 'alice%';
```

OR conditions on different columns:

```
-- May not use indexes efficiently:
SELECT * FROM users WHERE name = 'Alice' OR city = 'New York';

-- Better: Use UNION
SELECT * FROM users WHERE name = 'Alice'
UNION
SELECT * FROM users WHERE city = 'New York';
```


Real-World Example

E-commerce orders table:

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,           -- Automatic clustered index  
  user_id INTEGER NOT NULL,  
  product_id INTEGER NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  amount DECIMAL(10,2),  
  created_at TIMESTAMP DEFAULT NOW() );  
  
-- Index for finding user's orders (very common query)  
CREATE INDEX idx_orders_user_id ON orders(user_id);  
  
-- Index for product sales analysis  
CREATE INDEX idx_orders_product_id ON orders(product_id);  
  
-- Composite index for admin dashboard (common filter)  
CREATE INDEX idx_orders_status_created ON orders(status, created_at DESC);  
  
-- Partial index for active orders only (smaller, faster)  
CREATE INDEX idx_orders_active ON orders(user_id, created_at)  
WHERE status IN ('pending', 'processing');  
  
-- Covering index for order summary reports  
CREATE INDEX idx_orders_summary ON orders(created_at, status, amount);
```

Query performance:

```
sql  
  
-- Without indexes: 5000ms (full table scan of 10M rows)  
-- With idx_orders_user_id: 2ms  
  
SELECT * FROM orders WHERE user_id = 12345;
```

Best Practices

Start with primary and foreign keys These are your foundational indexes.

Index based on actual queries Don't guess—look at slow query logs and create indexes for real bottlenecks.

Use composite indexes wisely Order columns by selectivity (most selective first) or query patterns.

Monitor index usage Remove unused indexes periodically.

Don't over-index Every index slows down writes. Find the balance.

Keep indexes small Index fewer columns when possible. Smaller indexes = better cache hit rates.

Test with production-like data Index performance varies dramatically with data volume and distribution.

Consider cardinality High-cardinality columns (many unique values) benefit most from indexes.

Rebuild periodically Fragmentation degrades performance over time.

Indexing is about **trade-offs**: faster reads vs slower writes, more storage vs better performance. The goal is strategic indexing based on your actual query patterns, not indexing everything.